

Data types

```
In [ ]: A Data Types tells Python what kind of value is stored in a variable.
```

Real life example

```
In [ ]: Container1-> Rice  
Container 2 ->Sugar  
Container 3-> oil
```

```
In [ ]: a=10  
  
b=3.4  
  
c="Bhagyashri"  
  
d=3.4546677
```

Why Do we need data types

```
In [ ]: Data types helps:  
1.Store data efficiently.  
2.Perform the correct operations.  
3.Detect errors  
4.Manage memory properly
```

Data Types

```
In [ ]: Numeric:-int , float, complex  
  
text:-str  
  
Sequence:-list, tuple, range,set, frozenset  
  
mapping:-Dict  
  
Set:-Set, frozenset  
  
Boolean :-bool(True or False)
```

Numeric data types

1.int

```
In [ ]: int>> whole numbers

Positive or negative
20
30
-23
-29
0
100
```

```
In [7]: x=20
y=30
print(type(y))
print(type(x))

<class 'int'>
<class 'int'>
```

```
In [6]: type(x)
```

Out[6]: int

```
In [10]: x1=0
print(type(x1))

<class 'int'>
```

2.Float

```
In [ ]: Positive and negative number with decimal point.

3.14
6.78
-7.89
-9.45
-0.0
56789.3456
```

```
In [11]: a=10.56
print(f"value in variable a is {a} and data type is {type(a)}")

value in variable a is 10.56 and data type is <class 'float'>
```

```
In [12]: b=0.0000
print(type(b))

<class 'float'>
```

```
In [13]: c=-0.0001
print(type(c))

<class 'float'>
```

```
In [14]: a=20  
b=20.35  
a+b
```

Out[14]: 40.35

```
In [ ]: int*int>>int  
int+float>>float  
float+int>>float  
float+float>>float  
float-float>>float
```

```
In [ ]:
```

3.Complex

```
In [ ]: It contains real and imginanary _part  
number=real+imaginary_part
```

```
In [15]: a=3+5j  
type(a)
```

Out[15]: complex

```
In [16]: a=0+0j  
type(a)
```

Out[16]: complex

```
In [17]: a=-1-4j  
type(a)
```

Out[17]: complex

Python Type casting

```
In [ ]: To change datatype of variables
```

```
In [ ]: int>>float  
float>>int  
int>>complex  
float>>complex  
complex!>>int  
complex!>>float
```

```
In [19]: x=35  
type(x)
```

Out[19]: int

```
In [20]: x=25
y=float(x)
print(f"x=={x}")
print(f"y=={y}")
```

```
x==25
y==25.0
```

2. Float to int

```
In [22]: x=3.45
print(f"x=={x} Data type=={type(x)}")

y=int(x)
print(f"y=={y} data type =={type(y)}")
```

```
x==3.45 Data type==<class 'float'>
y==3 data type ==<class 'int'>
```

3.int to complex

```
In [23]: x=45
complex(x)
```

```
Out[23]: (45+0j)
```

```
In [25]: x=-5
y=complex(x)
print(type(y))
```

```
<class 'complex'>
```

4.float to complex

```
In [26]: x=-5.3
y=complex(x)
print(y)
```

```
(-5.3+0j)
```

```
In [27]: x=0.0
y=complex(x)
print(y)
```

```
0j
```

5. complex to int

can not convert complex to int

```
In [31]: x=4+5j
        y=int(x)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[31], line 2
      1 x=4+5j
----> 2 y=int(x)

TypeError: int() argument must be a string, a bytes-like object or a real number, not 'complex'
```

6. complex to float

```
In [ ]: can not convert complex to float
```

```
In [32]: x=4+6j
        y=float(x)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[32], line 2
      1 x=4+6j
----> 2 y=float(x)

TypeError: float() argument must be a string or a real number, not 'complex'
```

String

```
In [33]: s1="Python"
        type(s1)
```

```
Out[33]: str
```

int to string

```
In [36]: x=2345
        print(type(x))

        y=str(x)

        print(type(y))
```

```
<class 'int'>
<class 'str'>
```

float to string

```
In [37]: x=3.45
print(type(x))
y=str(x)

print(type(y))
```

```
<class 'float'>
<class 'str'>
```

###string to int

```
In [38]: s1='2345'
print(type(s1))

y=int(s1)
print(type(y))
```

```
<class 'str'>
<class 'int'>
```

```
In [40]: s2='23345a'
print(type(s2))

y=int(s2)
print(type(y))
```

```
<class 'str'>
```

ValueError

Traceback (most recent call last)

Cell In[40], line 4

```
1 s2='23345a'
2 print(type(s2))
----> 4 y=int(s2)
      5 print(type(y))
```

ValueError: invalid literal for int() with base 10: '23345a'

string to int

```
In [42]: s1="65"
a=int(s1)
print(a)
```

65

```
In [43]: s1="65"  
a=int(s1)  
print(type(a))
```

```
<class 'int'>
```

```
In [45]: s2="23456e"  
y=int(s2)  
print(type(y))
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[45], line 2  
      1 s2="23456e"  
----> 2 y=int(s2)  
      3 print(type(y))
```

```
ValueError: invalid literal for int() with base 10: '23456e'
```

string to float

```
In [46]: x="23.456"  
print(type(x))  
  
y=float(x)  
print(type(y))
```

```
<class 'str'>  
<class 'float'>
```

```
In [47]: x="23.456- "  
print(type(x))  
  
y=float(x)  
print(type(y))
```

```
<class 'str'>
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[47], line 4  
      1 x="23.456- "  
      2 print(type(x))  
----> 4 y=float(x)  
      5 print(type(y))
```

```
ValueError: could not convert string to float: '23.456- '
```

```
In [ ]:
```

